

Mutual Hazard Networks con Evamtools y Python - Grupo 6

Tania Gonzalo Santana
Claudia Guerrero Rodríguez
Sandra Mingo Ramírez
Daniel Parra Gutiérrez
2024/25

Índice general

1	Introducción a Mutual Hazard Networks	2
2	Comparación de implementaciones entre R y Python	2
2.1	Funcionalidad	2
2.2	Velocidad	4
2.3	Ventajas y desventajas	4
3	Integración del código en Python con evamtools	4
4	Modificar evamtools para incluir la corrección del sesgo	4

1. Introducción a Mutual Hazard Networks

Los modelos de progresión en cáncer (CPM) son métodos utilizados para predecir los futuros estados de un sistema y entender las restricciones en el orden de los eventos acumulados durante la progresión del tumor [1].

Entre esos modelos se encuentran los Mutual Hazard Networks (MHN), algoritmos de aprendizaje automático. Modelan la tasa de aparición de un suceso, como puede ser una mutación, mediante una tasa espontánea de adquisición y un evento multiplicativo que cada uno de estos sucesos puede tener sobre la tasa de otros sucesos mediante interacciones por pares que modelan dependencias promotoras e inhibidoras [2]. Eso representa la principal diferencia con los demás modelos de CPM; en MHN, los eventos no son determinísticos, si no que pueden afectar a la probabilidad de aparición de otros. Un supuesto del modelo es que el momento en que se produce una observación (por ejemplo, un diagnóstico de cáncer) es independiente de los acontecimientos precedentes. En su lugar, se supone que los tiempos de observación siguen una distribución exponencial simple, que es un supuesto estadístico estándar que a menudo se hace por simplicidad. No obstante, los MHN se han modificado recientemente para corregir el sesgo por observación [3]. Así, el acto de observar un acontecimiento se considera parte del proceso que se está modelando. Esto significa que el momento de una observación depende de los acontecimientos que hayan tenido lugar antes. Por ejemplo, una acumulación más agresiva de acontecimientos relacionados con el cáncer podría conducir a una detección más temprana.

Explicación matriz theta

2. Comparación de implementaciones entre R y Python

2.1. Funcionalidad

2.1.1. Python: MHN

El módulo `mhn` de Python cuenta con las funciones originales de MHN. Cuenta con implementación en CUDA, la GPU de Nvidia, para poder acelerar el cálculo de la puntuación de la log-verosimilitud y su gradiente tanto para el espacio de estados completo como para el restringido.

El módulo de Python cuenta con los dos frameworks comentados anteriormente, el clásico y el corregido. Los autores recomiendan en la mayoría de los escenarios utilizar el modelo con el sesgo corregido, ya que aumenta la interpretabilidad de los parámetros inferidos, aunque el modelado de los datos no se vea afectado. El primer paso es escoger la función de optimizado, siendo `cMHNOptimizer()` el del modelo MHN clásico y `oMHNOptimizer()` el del modelo corregido.

Se requiere que los datos de input estén en formato de matriz binaria, la cual se puede importar con el módulo `pandas` y la función `load_data_matrix()` o, en caso de que esté en un fichero csv, directamente con `load_data_from_csv()`.

Al tratarse de un algoritmo de aprendizaje automático, se requiere penalizar la inferencia de parámetros de modo que, en general, se prefiera un conjunto de parámetros disperso/parsimonioso. El resultado es explicar bien los datos de entrada de entrenamiento con el menor número posible de parámetros para que el modelo siga siendo generalizable a los datos de prueba. Así, los modelos generados son más sencillos e interpretables, evitando el sobreajuste. Para este paso, los autores han establecido dos formas de penalización:

- **Penalización estándar L1:** penaliza los parámetros individuales, fomentando que muchos de ellos sean exactamente cero. Esto conduce a un modelo disperso en el que sólo permanecen activos los parámetros más importantes.
- **Penalización simétrica personalizada:** penaliza los pares de parámetros que son simétricos en la matriz Θ (con respecto a la diagonal). El coste de que ambos parámetros de un par sean distintos

de cero equivale a que sólo uno sea distinto de cero. Al fomentar la simetría en el modelo, se puede alinear mejor con las relaciones biológicas, como los efectos epistáticos.

En el artículo [3], los autores recomiendan elegir la penalización basándose en el conocimiento previo sobre la estructura de los efectos que se esperan en el tipo de datos con los que se está trabajando.

El siguiente paso es la elección de hiperparámetros de validación cruzada. Se prueba un rango de diferentes penalizaciones («lambdas») para identificar una que produzca un rendimiento óptimo en los datos retenidos. Se pueden definir los siguientes hiperparámetros: la intensidad mínima y máxima de la penalización («lambda») que se probará (usualmente se espera un lambda de $1/n$, siendo n el número de observaciones, por lo que se escogen un mínimo y máximo alrededor de ese valor), la cantidad de pasos (es decir, la longitud de la secuencia lambda) que hay que dar entre estos límites y la cantidad de folds en que se quieren dividir los datos para la validación cruzada. Se consigue alta precisión probando con un gran rango lambda con pequeños tamaños de paso y utilizando muchos folds, pero requiere más recursos informáticos y más tiempo de computación. Por el contrario, los resultados más rápidos utilizan un rango más estrecho, menos pasos y menos folds, reduciendo el cálculo y sacrificando la precisión a la hora de encontrar la lambda óptima. En el módulo, todo esto se puede especificar en la función `lambda_from_cv()`.

El último paso es entrenar el modelo final con el lambda óptimo obtenido de la validación cruzada. Esto se consigue con `train()`, y los resultados se pueden guardar en formato csv y un fichero log en formato json con `result.save()`. El output principal se puede visualizar con `result.plot()`, mostrando la matriz Θ logarítmica de forma predeterminada.

2.1.2. Web app: evamtools

2.1.3. R: evamtools

El paquete de evamtools de R cuenta con 10 funciones exportadas:

- `evam`: genera un objeto que debería contener información sobre los métodos utilizados.
- `every_which_way_data`:
- `ex_mixed_and_or_xor`:
- `examples_csd`: proporciona ejemplos de datos de tipo cross-sectional (sección transversal), que se pueden usar como entrada para probar las funcionalidades del paquete. Esto es útil para experimentar con el paquete sin tener que preparar datos propios.
- `plot_CPMs`:
- `plot_evam`: genera representaciones gráficas de modelos ajustados EvAM, visualizando transiciones entre genotipos en términos de probabilidades o tasas de transición. También permite personalizar el tipo de gráfica y los datos mostrados.
- `random_evam`:
- `runShiny`:
- `sample_CMPs`:
- `sample_evam`:

2.2. Velocidad

2.3. Ventajas y desventajas

3. Integración del código en Python con evamtools

4. Modificar evamtools para incluir la corrección del sesgo

Bibliografía

- [1] Ramon Diaz-Uriarte y Iain G. Johnston. *A picture guide to cancer progression and monotonic accumulation models: evolutionary assumptions, plausible interpretations, and alternative uses*. 2023. DOI: [10.48550/ARXIV.2312.06824](https://doi.org/10.48550/ARXIV.2312.06824).
- [2] Rudolf Schill et al. "Modelling cancer progression using Mutual Hazard Networks". En: *Bioinformatics* 36.1 (jun. de 2019), págs. 241-249. ISSN: 1367-4803. DOI: [10.1093/bioinformatics/btz513](https://doi.org/10.1093/bioinformatics/btz513). eprint: https://academic.oup.com/bioinformatics/article-pdf/36/1/241/48981322/bioinformatics_36_1_241.pdf.
- [3] Rudolf Schill et al. "Correcting for Observation Bias in Cancer Progression Modeling". En: *Journal of Computational Biology* 31.10 (oct. de 2024), 927-945. ISSN: 1557-8666. DOI: [10.1089/cmb.2024.0666](https://doi.org/10.1089/cmb.2024.0666).